Smart Parish System

Setup Commands and Defense Explanation Guide
Iglesia Filipina Independiente Maragondon

1. PowerShell install commands

These are the commands your friends can use when testing the system on their own Windows device. They need Node.js for the NextJS frontend, Python for the Flask backend, and FFmpeg for media conversion used by AI transcription.

Install core tools

```
# Run PowerShell as Administrator
winget install OpenJS.NodeJS.LTS
winget install Python.Python.3.11
winget install Gyan.FFmpeg
```

Check installation

```
node -v
npm -v
python --version
ffmpeg -version
```

If npm is blocked by PowerShell

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

# Or use npm.cmd instead of npm
npm.cmd install
npm.cmd run dev
```

2. Frontend setup commands

Run these commands inside the main project folder, for example backup-church.

```
cd C:\Users\biga\Desktop\jscode\backup-church
npm install
npx prisma generate
npx prisma db push

# Optional, if the project has a seed file for the admin account
npx prisma db seed

npm run dev
```

The frontend runs at http://localhost:3000.

3. Backend setup commands

Run these commands in a second PowerShell terminal. The backend should be inside the main project folder as church_backend.

```
cd C:\Users\biga\Desktop\jscode\backup-church\church_backend
pip install flask flask-cors flask-sqlalchemy docxtpl faster-whisper reportlab
python server.py
```

The backend runs at http://127.0.0.1:5000. The system needs two terminals running at the same time: one for NextJS and one for Flask.

4. Full startup checklist

Terminal	Command	Purpose
Terminal 1	cd C:\Users\biga\Desktop\jscode\backup-church npm run dev	Starts the NextJS frontend.
Terminal 2	cd C:\Users\biga\Desktop\jscode\backup-church\church_backend python server.py	Starts the Flask backend and API.

5. Important project files and folders

```
backup-church/
  src/
    app/
      login/
      register/
      admin/
      dashboard/
      api/auth/
  prisma/
    schema.prisma
    dev.db
  church_backend/
    server.py
    models.py
    instance/archive.db
    uploads/
      baptismal/
      marriage/
      death/
      confirmation/
      masses/
      transcriptions/
    generated/
```

6. System flow

The system has two main parts: the NextJS frontend and the Flask backend. NextJS displays the pages, while Flask handles archive records, requests, uploaded files, announcements, mass recordings, and AI transcription.

```
User/Admin opens website
-> NextJS displays page
-> Frontend sends request to API
-> Flask or NextJS API processes request
-> Database saves or returns data
-> Frontend updates page
```

7. How the databases work

The system uses SQLite because it is simple, local, and easy to demonstrate. There are two SQLite databases because authentication and church operations have separate responsibilities.

Database	Location	Used for
Prisma SQLite	prisma/dev.db	Login, registration, user profile, account roles.
Flask SQLite	church_backend/instance/archive.db	Archive records, certificate requests, mass records, announcements, transcriptions, audit logs.

Prisma User table

```
User
```

```
- id
- fullName
- email
- password
- role
- createdAt
```

Flask database tables

```
Document
- sacramental archive records
- metadata and uploaded scanned files

CertificateRequest
- user certificate requests
- request status and attached document file

MassRecord
- uploaded mass video/audio
- AI transcript, summary, transcript PDF

Announcement
- admin-created announcements visible to users

AuditLog
- important system actions such as uploads and updates
```

8. Why the database is set up like this

The database was separated based on system responsibility. The NextJS Prisma database handles authentication because login and registration belong to the web application layer. The Flask SQLite database handles church records, requests, files, and AI transcription because these features are connected to Python processing, file handling, and AI libraries.

SQLite was chosen because this is a capstone prototype and local deployment is easier for testing. It does not require a separate database server like MySQL or PostgreSQL. In a future production version, the same structure can be migrated to MySQL or PostgreSQL.

9. What to show if the professor asks to see the database

Option 1: Show API output in browser

```
http://127.0.0.1:5000/api/archive/documents
http://127.0.0.1:5000/api/requests
http://127.0.0.1:5000/api/masses
http://127.0.0.1:5000/api/announcements
http://127.0.0.1:5000/api/transcriptions
```

Option 2: Open SQLite database files

```
Use DB Browser for SQLite.

Open:
church_backend/instance/archive.db

Show tables:
document
certificate_request
mass_record
announcement
audit_log

For login users, open:
prisma/dev.db

Show table:
User
```

Option 3: Prisma Studio for auth database

```
npx prisma studio
```

10. How the API works

The API is the bridge between the frontend and the database. The frontend sends HTTP requests to the API, and the API either saves data, retrieves data, updates records, or returns files.

Certificate request API

```
POST /api/requests
GET /api/requests
PATCH /api/requests/<id>/status
PATCH /api/requests/<id>/attach-document
```

Mass and transcription API

```
GET /api/masses
POST /api/masses
PATCH /api/masses/<id>/status
POST /api/masses/<id>/transcribe-ai
GET /api/transcriptions
```

Announcement API

```
GET /api/announcements
POST /api/announcements
DELETE /api/announcements/<id>
```

11. How archiving works

```
Physical church record
-> Admin scans the document
-> Admin uploads scanned PDF/image
-> Admin encodes metadata
-> System saves file in uploads folder
-> System saves metadata in archive.db
-> Admin can search, view, print, and attach it to requests
```

Archive metadata

```
Person name
Sacrament type
Parish
Priest
Date of record
Book number
Page number
Certificate number
Document type
Uploaded file path
```

The system does not store scanned files directly inside the database. The actual file is stored in the uploads folder, and the database stores the file path. This keeps the database lighter and easier to manage.

12. How request processing works

```
User submits certificate request
-> Request is saved as Pending
-> Admin sees it in Incoming Requests
-> Admin approves or rejects
-> Admin searches archive record
-> Admin attaches document ID
-> System marks request as Ready
-> User opens My Requests
-> User downloads the attached document
```

```
Statuses:
Pending
Approved
Rejected
Ready
Released
```

The user does not directly access all church records. The admin controls approval and attaches only the correct document to the user request.

13. How login and register works

Registration flow

```
User registers
-> NextJS sends data to /api/auth/register
-> Password is hashed using bcrypt
-> User is saved in Prisma dev.db
-> Role is set as guest
```

Login flow

```
User enters email and password
-> NextJS sends data to /api/auth/login
-> System checks email
-> System compares password using bcrypt
-> System checks role
-> System creates JWT token
-> Token is stored in cookie
-> User is redirected based on role

Role routing:
admin -> /admin
guest -> /dashboard
```

Profile flow

```
Dashboard profile calls /api/auth/me
-> System reads JWT cookie
-> System finds user in Prisma database
-> Profile displays real fullName, email, role, and joined date
```

14. How the admin account works

Public users can only register as guest. Admin registration is disabled publicly to prevent normal users from creating admin accounts. The admin account is created manually or through a seed file by the developer.

```
Current admin account:
Email: admin@ifimaragondon.com
Password: Admin12345
Role: admin
```

15. How AI transcription works

```
Admin uploads mass video/audio
-> File is saved in uploads/masses
-> Admin clicks AI Transcribe
-> Flask gets the file path
-> FFmpeg converts video/audio to WAV
-> Faster Whisper transcribes the WAV file
-> System formats the transcript into paragraphs
-> System extracts highlight/summary
-> System creates PDF transcript
-> Transcript is saved in database
-> Mass status becomes Transcribed
-> User can view it in Transcribed Mass
```

Technical process

```
POST /api/masses/<id>/transcribe-ai
-> Find MassRecord by ID
-> Check uploaded file path
-> Convert to 16kHz mono WAV using FFmpeg
-> Run Faster Whisper model
-> Save transcript_text
-> Save summary_text
-> Generate transcript PDF
-> Update status to Transcribed
```

FFmpeg is used because it converts different audio/video formats into a clean WAV format that Whisper can process reliably. Faster Whisper is used because it can run locally and does not require a paid API. It is slower on CPU, so short 1-3 minute videos are best for demonstration.

16. How announcements work

```
Admin creates announcement
-> Announcement is saved in archive.db
-> User dashboard fetches announcements
-> Announcement appears on user dashboard and announcement page
```

Admin can publish and delete announcements. Users can view announcements only.

17. How mass archive works

```
Admin uploads mass recording
-> Mass record is saved in database
-> File is saved in uploads/masses
-> User Mass Archives page fetches /api/masses
-> User can view/open uploaded mass recording
-> After AI transcription, the mass appears in Transcribed Mass page
```

18. How to explain how you came up with the development

I designed the system based on the actual manual process of a parish. Church records are usually stored physically and searched manually. To improve this, I created a digital archive where scanned records are uploaded with searchable metadata. Then I added a request system so parishioners can request certificates without directly accessing sensitive records. The admin still controls approval, document attachment, and release. For Mass recordings, I added AI transcription because churches often record Masses and homilies, but manually transcribing them is time-consuming. Using Python, FFmpeg, and Faster Whisper allowed the system to generate transcripts and summaries locally without paid APIs.

19. Short defense summary

Our system is a Smart Parish System for Iglesia Filipina Independiente Maragondon. It has three major parts: digital archiving, certificate request processing, and AI Mass transcription. The user can register, login, submit certificate requests, view announcements, view Mass archives, and download approved documents. The admin can upload sacramental records, manage certificate requests, attach documents, upload Mass recordings, publish announcements, and run AI transcription. The system uses NextJS for the frontend, Prisma SQLite for authentication, and Flask SQLite for archive operations and AI processing. Files are stored in folders while metadata is stored in the database. AI transcription uses FFmpeg to convert media and Faster Whisper to generate transcripts, summaries, and PDF outputs.

20. Common professor questions

Question	Answer

Why Python?	Python was used for the backend archive and AI transcription because it has strong support for file handling, Flask APIs, SQLite, and AI libraries. Faster Whisper and ReportLab are easier to integrate in Python.
Why NextJS?	NextJS was used for the frontend because it supports React components, routing, dashboard pages, API routes for authentication, and organized UI development.
Is the system secure?	The system has basic security through password hashing, role-based login, admin-only access, and JWT cookies. Passwords are hashed using bcrypt. Future improvements include stronger middleware, backups, and stricter file validation.
What are the limitations?	The system is a local prototype. It uses local SQLite databases and local file storage. AI transcription depends on CPU speed, so long videos may take time. Future improvements include cloud backup, production hosting, email notifications, and stronger security middleware.

21. Final system summary

Guest/User side

- Register
- Login
- Profile with real account data
- Submit certificate requests
- View request status
- Download approved document
- View admin-created announcements
- View admin-uploaded Mass archives
- View AI-generated transcriptions and summaries

Admin side

- Login as admin
- Upload archive records
- Search and view archive files
- Manage incoming requests
- Approve/reject/ready/release requests
- Attach archive document to user request
- Upload Mass recordings
- Run AI transcription
- Publish announcements
- View dashboard stats

Technologies

```
Frontend: NextJS, React, TypeScript, Tailwind
Auth Database: Prisma + SQLite
```

```
Backend: Python Flask
Archive Database: Flask SQLAlchemy + SQLite
AI: Faster Whisper
Media conversion: FFmpeg
PDF generation: ReportLab
File storage: uploads/ and generated/
```